

Order Processing & Outbox

One local transaction • Reliable publication • The duplicate window

SAY IT IN THE INTERVIEW

"I save the order and its event in the same MySQL transaction. A scheduled publisher sends pending events to Kafka. I mark a row PUBLISHED only after a successful send. This protects the order-to-event handoff, but duplicates are still possible, so consumers must be idempotent."

01 WHY THIS PATTERN EXISTS

- **Transactional Outbox:** save business data and an event-to-publish in the same database transaction; publish it separately.
- Direct DB save + Kafka send is a **dual write**. One can succeed while the other fails. Our local transaction covers **orders + outbox**; it does not include Kafka.

02 WHAT MY CODE DOES

- **POST /api/orders:** create order with status **pending**; create ORDER_CREATED outbox row with status **PENDING**. @Transactional commits both or rolls back both.
- Publisher polls pending rows by created time; **fixedDelay=1000** means a delay after the previous run finishes, not a guaranteed event every second.
- Stored payload is JSON. Publisher converts it to Avro, sets **source=WEB**, and sends to **orders.created.avro** using **orderId** as the Kafka key.

03 CRASH WINDOWS — THE IMPORTANT FOLLOW-UP

- **Before DB commit:** neither durable order nor durable outbox entry.
- **After DB commit, before send:** row remains PENDING; the publisher can retry.
- **Kafka accepts, process dies before PUBLISHED save:** the same event can be sent again. Preserve its eventId; see Chapter 04 for consumer protection.
- Successful send → save PUBLISHED + publishedAt. A failed send is not treated as successful publication.

04 TIMEOUTS & HONEST LIMITS

- Registry HTTP connect/read: **60s**; Kafka delivery: **120s**; publisher future wait: **130s**. The future wait starts after publish() returns; synchronous serialization can add time before it.
- Failure/timeout keeps the durable row pending; interruption also restores the thread interrupt flag. Timeout can mean an **unknown send outcome**, so retries may duplicate.
- Current publisher has no row-claiming lease for multiple instances. This is polling outbox, not CDC; published-row retention is a future operational concern.

YAAD RAKHO: DB commit ≠ Kafka publish; Kafka publish ≠ outbox status save.